# MiniProject 1 Ordered Student List

Engineering Data Structures | Implementation and test report | October 21, 2026

## Purpose and implementation

The program stores student records in a templated doubly linked list ordered by M Number. It supports traversal, removal, independent copying, reversal, and text-file storage. The existing Student class, node structure, and console menu form the basis of the implementation.

## Tasks 1 and 2

DoubleyLinkedList<T> owns its nodes and successfully inserted objects. AddItem inserts in sorted order; GetItem removes a matching object and transfers ownership to the caller. SeeNext starts at the first item, returns each item once, and returns null past the end. SeeAt uses zero-based indices and advances the cursor to the following item. Reset returns the cursor to the head. Removing the pending item advances the cursor safely.

Student stores first and last names, M Number as a string, GPA, and birthday. GPA defaults to zero. Age is computed from the current date. The comparison operators compare two students by M Number. M Numbers are validated as uppercase M followed by eight digits, using the format in the assignment example.

## Tasks 3 and 4

The menu exposes every required list operation. Student display uses ASCII boxes and reuses the name, M Number, and age accessors. It shows name for identification, M Number as the unique key, GPA for academic information, and age as a readable summary of birthday. GPA zero remains visible. DisplayAll calls each student display method without changing the traversal cursor.

## Tasks 5 and 6

Copy construction allocates independent students and nodes and preserves the source cursor position. Assignment builds a separate copy and swaps it into place, releasing the previous contents and handling self-assignment safely. RemoveAll deletes every owned object and node. Largest and smallest return borrowed pointers without removal. Reverse swaps each node link and exchanges head and tail. It resets traversal and toggles the insertion direction, so later additions preserve the current ordering.

## Task 7

Saving writes quoted names, M Number, birthday, and GPA, one record per line. Loading validates an independent temporary list and then replaces the current contents, sorting ascending. A missing or malformed file leaves current data intact. Invalid names, IDs, GPAs, birthdays, and duplicates are rejected. Numeric-input errors do not trap the menu in a failed input state.

## Test results

The project and the submission entry point compiled using GCC 13.1.0 in C++20 mode. Warning checks with -Wall -Wextra -Wpedantic produced no warnings. CTest passed its list test executable. The following image renders the actual captured test output.

```
PASS: sorting, empty/single lists, SeeNext, SeeAt and Reset
PASS: removal, independent copies, assignment and self-assignment
PASS: Reverse, extremes, DisplayAll and cursor preservation
PASS: save/load, empty file, missing file, corrupt/duplicate records
PASS: names, M Numbers, GPA, birthdays, duplicates and serialization
PASS: generic items and object lifetime accounting
All tests passed.
```

## Storage and menu tests

A separate menu test saved an empty list, added students out of order, used the default GPA, saved, exited, started a new process, and reloaded the saved file. It verified restored count and ascending order. Additional runs verified duplicate rejection, GPA rejection, nonexistent-file errors, invalid menu input recovery, and clean exit on end of input. All checks passed.

## Memory checks

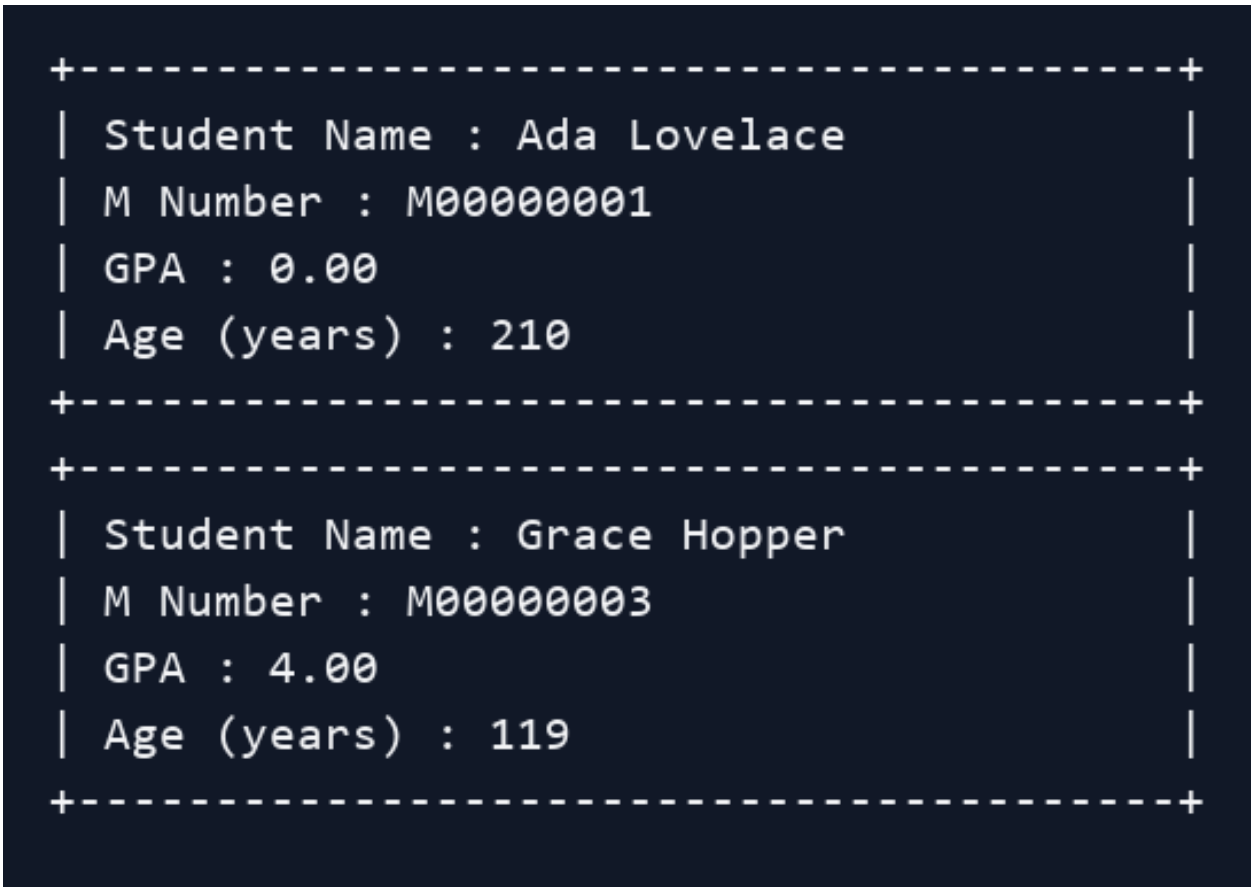
The tests exercise removal of head, middle, tail, and the only node; removal at the traversal cursor; empty and populated assignment; self-assignment; and destruction. A second template item type counts live objects and verifies a zero count after copying, assignment, removal, clearing, and scope exit. This validates owned object cleanup for those scenarios; a full heap profiler was not run.

## Captured validation output

```
Error: A student with M00000001 already exists.
Student was not added.
Error: GPA must be between 0.0 and 4.0.
Error: Unable to open file or invalid student data.
Error: Please enter a valid number.
```

## Student visualization

The image below renders the actual display output captured from the menu test. Ada was added after Grace; the display correctly sorts Ada first. A blank GPA entry produced the displayed value 0.00. Ages reflect the execution system date.




## Build and submission

Open the folder in CLion using its bundled MinGW toolchain, or run cmake -S . -B build followed by cmake --build build. Run ctest --test-dir build --output-on-failure for list tests. The executable is build/MiniProject1_EECE2028C.exe with a single-configuration Windows build.

Alternatively, compile g++ -std=c++20 Miniproject1.cpp -o MiniProject1. Submit that entry point together with src/ and include/, since it includes the implementation files. Do not compile the entry point alongside src/main.cpp. The original root executable is stale; run a fresh build.

The sample input file docs/sample_students.txt contains three intentionally unsorted students. Load it using menu option 8, display with option 4, and save with option 7. README.md documents ownership, file format, and commands. Optional menu tests run with python tests/menu_tests.py build/MiniProject1_EECE2028C.exe.